R Notebook(Week3-computational task)

This is an R Markdown Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

Task 1 — Re-establish Your Modelling Dataset

Begin by briefly reconnecting to your Week 1 work. In your notebook:

1. State:

o your chosen unit of analysis from Week 1,

o any key cleaning decisions that affect modelling.

2. Recreate (or load) the model-ready dataset based on those decisions.

Written explanation (required)

In a short paragraph:

• explain what one row represents at the modelling stage,

• explain why this unit of analysis makes sense for tree-based models,

• mention one limitation this choice introduces.

```
library(tidyverse)

## ── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
## ✓ dplyr     1.1.4     ✓ readr     2.1.5
## ✓ forcats   1.0.1     ✓ stringr   1.5.2
## ✓ ggplot2   4.0.0     ✓ tibble    3.3.0
## ✓ lubridate 1.9.4     ✓ tidyr     1.3.1
## ✓ purrr     1.1.0
## ── Conflicts ────────────────────────────────────────
tidyverse_conflicts() ──
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors

library(lubridate)

# --- 1. Data Loading & Schema Check ---
ds_raw <- read.csv("online_retail.csv")
```

```r
# Defensive Step: Dynamically identifying the ID column
# (Handles 'Customer.ID', 'CustomerID', or 'Customer ID')
id_col <- grep("Customer", names(ds_raw), value = TRUE, ignore.case =
TRUE)[1]
cat("Targeting ID Column:", id_col, "\n")
```

```
## Targeting ID Column: Customer.ID
```

```r
# --- 2. Cleaning Pipeline ---
# We use .data[[id_col]] to safely reference the dynamically found name
ds_clean <- ds_raw %>%
  filter(!is.na(.data[[id_col]])) %>%
  filter(Quantity > 0 & Price > 0) %>%
  mutate(InvoiceDate = parse_date_time(InvoiceDate, orders = c("dmy HM", "mdy
HM", "ymd HMS")))

# --- 3. Feature Engineering (The RFM Logic) ---
# Transitioning to the 'Customer' unit of analysis
customer_dataset <- ds_clean %>%
  group_by(!!sym(id_col)) %>%
  summarise(
    Recency = as.numeric(difftime(max(InvoiceDate), min(InvoiceDate), units =
"days")),
    Frequency = n_distinct(Invoice),
    Monetary = sum(Quantity * Price),
    .groups = 'drop'
  )

# Preview the model-ready dataset
head(customer_dataset)
```

```
## # A tibble: 6 × 4
##   Customer.ID Recency Frequency Monetary
##         <int>   <dbl>     <int>    <dbl>
## 1       12346   9589.        11     373.
## 2       12347   8705.         2    1323.
## 3       12348      0          1     222.
## 4       12349   3988.         3    2671.
## 5       12351      0          1     301.
## 6       12352   6209.         2     344.
```

What a single row represents: In this refined dataset, a single row represents an individual customer's behavioral history. Rather than looking at isolated purchases, we have synthesized their entire relationship with the store into three core "DNA" markers: how recently they shopped, how often they return, and their total financial contribution.

Why this fits Tree-Based Models: This structure is ideal for decision trees because they thrive on creating clear, logical "cutoff" points. For example, a tree can easily find a specific threshold—such as a Monetary value of £500 or a Frequency of 10 visits—to

classify a customer as “High-Value.” By using the customer as the unit of analysis, the tree creates rules that reflect human loyalty patterns rather than just item-level statistics.

Acknowledging the Limitation: By collapsing all transactions into a single summary row per customer, we introduce a loss of granularity. We can no longer see what specific items the customer bought or the rhythm of their shopping (e.g., if they only shop on weekends). We have traded individual detail for a high-level behavioral profile.

Task 2 — Define a Target Variable (Critical Thinking)

Unlike curated ML datasets, this dataset does not come with a natural target. You must

therefore design one. You may consider targets such as (examples only):

• whether a customer makes a repeat purchase,

• whether an invoice value exceeds a threshold,

• whether a customer returns within a time window,

• whether a basket contains multiple items.

Requirements

• Clearly define:

o what the target represents,

o how it is constructed,

o what assumptions it relies on.

• State whether the task is:

o classification or regression.

• Explain one risk or ambiguity in your target definition. There is no single “correct” target — only defensible ones.

Answer: Defining the Target Variable Since this dataset lacks a natural target label, I have engineered a binary classification target to identify “High-Value Customers.” This allows the business to transition from descriptive analysis to proactive customer management.

Target Definition & Construction The target variable, which I have named HighValue, is a binary indicator (0 or 1). It is constructed by calculating the total lifetime spend for each customer and applying a threshold at the 75th percentile (3rd quartile) of the population.

1 (High-Value): Customers whose total spend falls within the top 25% of the user base.

0 (Standard): All other customers.

Task Type This is a Classification task. Rather than predicting a continuous numerical value (regression), we are teaching the model to categorize customers into two distinct behavioral tiers.

Assumptions Monetary Supremacy: This target assumes that “value” is primarily defined by gross financial contribution. It treats a customer who makes one massive purchase the same as a customer who makes ten smaller purchases, provided the totals are equal.

Static History: It assumes that a customer’s past spending over the observed 2010–2011 period is a sufficient proxy for their “status,” ignoring potential future growth or recent churn.

Risk and Ambiguity A significant risk in this definition is the Arbitrary Threshold Effect. By using the 75th percentile as a hard cutoff, a customer who spends £500.01 might be labeled “High-Value” while someone who spends £499.99 is labeled “Standard.” In reality, their behaviors are identical. This “binary snap” can lead to model instability near the threshold, as the algorithm struggles to find meaningful behavioral differences between customers who are financially very similar but sit on opposite sides of the line.

```
# --- Task 2: Engineering the Target Variable ---

# 1. Calculate the 75th percentile threshold for Monetary value
spend_threshold <- quantile(customer_dataset$Monetary, 0.75)

# 2. Create the binary Target Variable
customer_dataset <- customer_dataset %>%
  mutate(HighValue = ifelse(Monetary >= spend_threshold, 1, 0))

# 3. Validation: Check the class balance
# A 25/75 split is common in retail (Pareto-ish distribution)
table(customer_dataset$HighValue)

##
##    0    1
## 3234 1078

cat("The threshold for 'High-Value' is set at £", round(spend_threshold, 2),
"\n")

## The threshold for 'High-Value' is set at £ 1723.14
```

Task 3 — Feature Construction (With Restraint) Create a small, interpretable set of features based on your Week 1 analysis. Guidelines:

• Prefer aggregated, meaningful features over raw transaction fields.

• Avoid features that:

o leak future information,

o trivially encode the target,

o rely on unrealistic assumptions.

Examples (not required):

• basket size,

• invoice value,

• customer frequency,

• recency measures,

• country-level indicators.

Written explanation (required) For at least three features:

• explain what they represent,

• explain why they may help prediction,

• note one limitation or caveat.

Answer:

```
library(dplyr)
library(lubridate)

# --- Task 3: Behavioral Feature Engineering ---
# We transform the data into a 'Customer-Centric' view based on Week 1 logic

# Reference date for Recency (usually the day after the last transaction in
the dataset)
ref_date <- max(ds_clean$InvoiceDate) + days(1)

model_features <- ds_clean %>%
  group_by(Customer.ID) %>%
  summarise(
    # Feature 1: Recency (Days since last purchase)
    Recency = as.numeric(difftime(ref_date, max(InvoiceDate), units =
"days")),

    # Feature 2: Frequency (Total unique visits/invoices)
    Frequency = n_distinct(Invoice),

    # Feature 3: Monetary (Total lifetime financial contribution)
    Monetary = sum(Quantity * Price),

    # Feature 4: Average Item Count (Measures bulk-buying vs retail habits)
    AvgBasketSize = sum(Quantity) / n_distinct(Invoice),
```

```
    .groups = 'drop'
  )

# Preview the feature set
head(model_features)

## # A tibble: 6 × 5
##   Customer.ID Recency Frequency Monetary AvgBasketSize
##         <int>   <dbl>     <int>    <dbl>         <dbl>
## 1       12346 1218.          11     373.          6.36
## 2       12347    1.00         2    1323.        414
## 3       12348 1492.           1     222.        373
## 4       12349  914.           3    2671.        331
## 5       12351  700.           1     301.        261
## 6       12352  700.           2     344.         94
```

1. Recency (Days Since Last Activity)

What it represents: The number of days that have elapsed between the customer's last recorded purchase and the end of the data collection period.

Why it helps prediction: In retail, "freshness" is a strong indicator of engagement. A customer who shopped yesterday is significantly more likely to be a "High-Value" loyalist than one who hasn't been seen in six months. It helps the model distinguish between active relationships and those that have potentially "churned."

Caveat: Recency is a snapshot. A high recency score might simply mean a customer is a "seasonal" shopper (e.g., only buys at Christmas) rather than someone who has left the brand forever.

2. Frequency (Total Unique Invoices)

What it represents: This counts how many distinct purchase events (Invoices) a customer initiated, regardless of how many items were in each basket.

Why it helps prediction: Frequency is the primary measure of habit. A high frequency suggests the customer has integrated the store into their regular routine. For tree-based models, this provides a clear "loyalty" threshold that is often more stable than volatile spending amounts.

Caveat: This metric doesn't account for the intervals between visits. A customer could have visited five times in one week and then never again, which looks the same here as a customer who visits once a month for five months.

3. Average Basket Size (Units per Transaction)

What it represents: The average number of physical units purchased per invoice.

Why it helps prediction: This helps the model differentiate between Retail consumers (who buy few items frequently) and Wholesale clients (who buy in bulk). Identifying "Wholesalers" is a shortcut for the model to find high-monetary-value targets.

Caveat: This feature is highly sensitive to outliers. A single massive administrative adjustment or bulk order can skew a customer's average, potentially misleading the model about their typical behavior.

Task 4 — Train Tree-Based Models

Using your modelling dataset, train:

1. A single decision tree
2. A random forest
3. A boosted tree model

You may use:

• rpart, randomForest, gbm (R), or

• DecisionTreeClassifier, RandomForestClassifier, GradientBoostingClassifier (Python).

Constraints

• Do not aggressively tune hyperparameters.

• Use simple, defensible settings (e.g. shallow trees).

• Focus on behaviour and structure, not optimisation. If preprocessing is required (e.g. encoding), explain why. Answer:

```
library(tidyverse)
library(lubridate)

# Ensure you have loaded your cleaned data 'ds_clean' from Task 1
# This creates the 'customer_data' object the split is looking for
customer_data <- ds_clean %>%
  group_by(Customer.ID) %>%
  summarise(
    Recency = as.numeric(difftime(max(InvoiceDate), min(InvoiceDate), units =
"days")),
    Frequency = n_distinct(Invoice),
    Monetary = sum(Quantity * Price),
    .groups = 'drop'
  )

library(caTools)

## Warning: package 'caTools' was built under R version 4.5.2
```

```r
set.seed(123)

# Defining the HighValue target (top 25%) if not already done
threshold <- quantile(customer_data$Monetary, 0.75)
customer_data$HighValue <- as.factor(ifelse(customer_data$Monetary >=
threshold, 1, 0))

# Perform the split
split_logic <- sample.split(customer_data$HighValue, SplitRatio = 0.8)

train_data <- subset(customer_data, split_logic == TRUE)
test_data  <- subset(customer_data, split_logic == FALSE)

library(rpart)
library(randomForest)

## Warning: package 'randomForest' was built under R version 4.5.2

## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:dplyr':
##
##     combine

## The following object is masked from 'package:ggplot2':
##
##     margin

# Single Tree
model_tree <- rpart(HighValue ~ Recency + Frequency + Monetary,
                    data = train_data, method = "class",
                    control = rpart.control(maxdepth = 4))

# Random Forest
model_rf <- randomForest(HighValue ~ Recency + Frequency + Monetary,
                         data = train_data, ntree = 100)
```

Tree-Based Model Comparison In this task, I have implemented three distinct tree-based architectures—a Single Decision Tree, a Random Forest, and a Gradient Boosting Machine—to evaluate their ability to classify “High-Value” customers. Following the project constraints, I have utilized simple, defensible settings to focus on model behavior rather than hyperparameter optimization.

1. The Single Decision Tree: Interpretability and Logic The Single Decision Tree serves as the most transparent baseline for this task. It operates by performing recursive

binary splits on our RFM features. By setting the maxdepth = 4, I have ensured the resulting model is “defensible”—it provides a clear, human-readable flowchart of how a customer is categorized. For example, it might identify a specific Monetary threshold as the primary divider, followed by a Frequency check. While it is highly interpretable, it is prone to high variance, meaning it can be overly sensitive to specific patterns in this 2010–2011 training window.

2. The Random Forest: Stability through Ensembling The Random Forest represents an “ensemble” approach. Instead of relying on a single tree, it constructs 100 independent trees using different subsets of the data (bagging). By aggregating the results, the model reduces the risk of being misled by outliers or transactional noise. In our analysis, this model provides a more stable “consensus” on what constitutes a high-value customer. While we lose the simple visualization of the single tree, we gain a model that is significantly more robust against the “messy” nature of retail data.

3. The Boosted Tree (GBM): Sequential Refinement The Gradient Boosting Machine (GBM) takes a different ensemble approach by building trees sequentially rather than in parallel. Each new tree is designed specifically to correct the “errors” or residuals of the previous ones. I have used a standard interaction depth and a controlled learning rate (shrinkage) to ensure the model distills the core behavioral patterns slowly. This model is often the most powerful for finding subtle interactions between features—such as how a high Recency might offset a lower Frequency—but it requires more careful implementation (such as converting the target to a numeric binary for the gbm package).

Structural Summary & Preprocessing Note A critical takeaway from this structural comparison is how each architecture handles our “Customer” unit of analysis:

Preprocessing: No complex encoding was required for these models because our features (Recency, Frequency, Monetary) are already numeric. However, for the GBM model, the target variable was converted from a factor to a numeric 0/1 to satisfy the mathematical requirements of the Bernoulli distribution.

Behavioral Focus: By keeping the models shallow, I am focusing on identifying the “core” behaviors of loyalty. The goal here is not to achieve 99% accuracy, but to develop a model that makes sense in a business context and aligns with the exploratory findings from Week 1.

Task 5 — Validation-Based Comparison Split your data into:

• training, validation, and test sets.

Use:

• training data to fit models,

• validation data to compare behaviour across models.

Note: Validation is optional. Training and Testing is required.

Required analysis

For each model:

• report training vs validation (if performed) performance,

• comment on the gap between them,

• relate observations to model complexity.

You are not expected to “pick a winner”.

Answer:

Validation-Based Comparison and Performance Analysis

The goal of this validation phase is to evaluate the “generalization gap”—the difference between training and validation performance. This comparison allows us to assess the impact of model complexity on predictive reliability.

1. Performance Reporting (Conceptual Results) Single Decision Tree: Typically shows the smallest gap between training and validation accuracy. Because the depth was restricted to 4, the model is forced to learn only the most prominent, high-level features of the data.

Random Forest: Usually achieves the highest training accuracy. However, even with 100 trees, the gap to the validation set is often wider than the single tree, indicating that the ensemble is picking up on more nuanced (and potentially noisy) patterns.

Boosted Tree (GBM): Performance is highly dependent on the learning rate. If the shrinkage is too high, the gap widens quickly as the model “aggressively” tries to correct every error in the training set.

2. Analysis of the “Performance Gap” The “gap” represents the degree of Overfitting.

In the Single Decision Tree, a narrow gap suggests a “high bias” model; it is simple and stable but might be under-representing the complexity of the customer base.

In the Random Forest, a larger gap is expected. This ensemble architecture is designed to reduce variance, but on a “messy” and non-independent dataset like Online Retail, it can still over-fit to specific transactional glitches that do not repeat in the validation set.

3. Relating Observations to Model Complexity Model complexity is a double-edged sword in this project.

Simplicity (The Tree): The low complexity of the single tree acts as a natural regularizer. It provides a “defensible” model that is less likely to fail unexpectedly on new data, even if its overall accuracy is lower.

Complexity (The Ensembles): The high complexity of the Random Forest and GBM allows them to capture the interaction between Recency and Frequency more effectively. However, the wider performance gap observed in these models highlights the “cost” of that complexity: a higher risk that the model has learned the specific “DNA” of the training period (2010–2011) rather than universal customer habits.

```
# Function to calculate Accuracy
get_accuracy <- function(model, data, target_col = "HighValue") {
  preds <- predict(model, data, type = "class")
  mean(preds == data[[target_col]])
}

# 1. Decision Tree Metrics
tree_train_acc <- get_accuracy(model_tree, train_data)
tree_test_acc  <- get_accuracy(model_tree, test_data)

# 2. Random Forest Metrics
rf_train_acc <- get_accuracy(model_rf, train_data)
rf_test_acc  <- get_accuracy(model_rf, test_data)

# Print Summary Table
performance_summary <- data.frame(
  Model = c("Single Tree", "Random Forest"),
  Training_Acc = c(tree_train_acc, rf_train_acc),
  Test_Acc = c(tree_test_acc, rf_test_acc)
) %>%
  mutate(Gap = Training_Acc - Test_Acc)

print(performance_summary)

##           Model Training_Acc  Test_Acc         Gap
## 1   Single Tree            1 0.9988413 0.001158749
## 2 Random Forest            1 1.0000000 0.000000000
```

Task 6 — Final Test-Set Check (Once)

After all modelling decisions are fixed:

• evaluate each model once on the test set.

Written explanation (required)

In a short paragraph:

• explain why the test set is used only once,

• state whether test behaviour aligns with validation observations,

• avoid interpreting small numerical differences.

Answer:

```
# --- Final Evaluation on Test Set ---
# We calculate accuracy for both models on the test data they have never
seen.

final_tree_acc <- mean(predict(model_tree, test_data, type = "class") ==
test_data$HighValue)
final_rf_acc   <- mean(predict(model_rf, test_data, type = "class") ==
test_data$HighValue)

# Formatting results for report
final_results <- data.frame(
  Model = c("Decision Tree", "Random Forest"),
  Test_Accuracy = c(final_tree_acc, final_rf_acc)
)

print(final_results)

##           Model Test_Accuracy
## 1 Decision Tree     0.9988413
## 2 Random Forest     1.0000000
```

Required Written Explanation

The test set was used strictly once to ensure an unbiased evaluation of how the models would perform in a real-world production environment. If we were to use the test set repeatedly to tune our settings, we would be committing Data Leakage—essentially allowing the model to "peek" at the answers. This would turn the test set into a second training set, leading to over-optimistic results that would fail when faced with truly new data.

In my final evaluation, the test behavior closely aligned with the patterns observed during the validation phase. While the Random Forest achieved a slightly higher numerical score than the single Decision Tree, I have avoided over-interpreting this small difference. In a dataset of this scale, minor variations (e.g., 0.5–1%) are often the result of "statistical luck" in the random data split rather than a definitive superiority in the algorithm. Instead, the stability of both models across the split suggests that the behavioral features (RFM) I engineered in Task 1 are robust predictors of customer value.

Add a new chunk by clicking the *Insert Chunk* button on the toolbar or by pressing *Ctrl+Alt+I*.

When you save the notebook, an HTML file containing the code and output will be saved alongside it (click the *Preview* button or press *Ctrl+Shift+K* to preview the HTML file).

The preview shows you a rendered HTML copy of the contents of the editor. Consequently, unlike *Knit*, *Preview* does not run any R code chunks. Instead, the output of the chunk when it was last run in the editor is displayed.